CENTRO UNIVERSITÁRIO JORGE AMADO - UNIJORGE
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Cristian Souza Santos, Breno José dos Santos Neri , Maria Luiza Marins Barbosa, Yuri Santos Do Carmo Cardoso

PROJETO PRÁTICO: OTIMIZAÇÃO DE CÓDIGO
AV2 - DIAGNÓSTICO E PRIMEIRA OTIMIZAÇÃO

SALVADOR
2026

Cristian Souza Santos, Breno José dos Santos Neri , Maria Luiza Marins Barbosa, Yuri Santos Do Carmo Cardoso

PROJETO PRÁTICO: OTIMIZAÇÃO DE CÓDIGO
AV2 - DIAGNÓSTICO E PRIMEIRA OTIMIZAÇÃO

Trabalho acadêmico (AV2) apresentado à disciplina de Análise e Otimização de Sistemas do curso de Ciência da Computação da UNIJORGE, como requisito parcial de avaliação.

Orientador: Prof. Igor Gonzalez

SALVADOR

2026

1. DESCRIÇÃO DO PROBLEMA

O problema escolhido é a detecção de valores duplicados, em uma lista. Dada uma sequência de elementos, a função deve retornar True quando um mesmo valor aparece mais de uma vez e False quando todos os valores são distintos. A saída deve permanecer igual à do código-base para as mesmas entradas.

O enunciado apresenta como ponto de partida uma solução que compara todos os pares de posições. Essa estratégia é simples e correta, mas tem um problema que é o ato de repetir comparações e se torna custosa à medida que a lista cresce. O objetivo da AV2 é medir esse comportamento e aplicar uma primeira melhoria usando uma estrutura de dados adequada.

2. ESCOPO E OBJETIVO DA AV2

O escopo do trabalho engloba os seguintes tópicos fundamentais:

• Manter a função original como referência para comparação;

• Implementar uma versão otimizada com hashing por meio de set;

• Comparar os resultados para garantir equivalência funcional;

• Medir os tempos com listas de tamanhos crescentes e apresentar evidências.

3. VERSÃO ORIGINAL E GARGALO

A implementação original utiliza assim dois laços aninhados! Para cada posição i, compara o elemento com todas as posições posteriores. Quando a lista não contém duplicatas, nenhuma comparação encerra a busca antecipadamente: todos os pares são examinados.

```
for i in range(len(lista)):
    for j in range(i + 1, len(lista)):
        if lista[i] == lista[j]:
            return True
return False
```

O gargalo está no número de comparações, que cresce quadraticamente com n.

4. ANÁLISE DE COMPLEXIDADE

No pior caso (lista sem repetição), a função compara todos os pares de elementos. A quantidade de comparações é $(n - 1) + (n - 2) + ... + 1 = n(n - 1)/2$. O termo dominante é n^2,

portanto a complexidade de tempo é $O(n^2)$. A versão original utiliza apenas variáveis de controle e não cria uma estrutura proporcional à entrada, resultando em O(1) de espaço auxiliar.

5. PRIMEIRA OTIMIZAÇÃO: ESTRUTURA SET

A versão otimizada percorre a lista uma única vez e mantém um conjunto chamado vistos. Para cada elemento, verifica se ele já está no conjunto. Se estiver, existe uma duplicata e a função retorna True; caso contrário, insere o elemento e continua. Operações de busca e inserção em set têm custo esperado O(1), de modo que a passagem completa tem custo esperado O(n).

```
vistos = set()
for item in lista:
    if item in vistos:
        return True
    vistos.add(item)
return False
```

A implementação entregue também preserva a compatibilidade com elementos não hashable: quando a inserção no conjunto gera TypeError, a função recorre à versão original. Assim, listas contendo sublistas continuam sendo avaliadas corretamente, embora esse caminho de fallback tenha custo $O(n^2)$.

Tabela 1 – Resumo de complexidade dos algoritmos analisados

Versão	Tempo (pior / esperado)	Espaço auxiliar	Observação
Original	$O(n^2)$ no pior caso	O(1)	Compara todos os pares
Otimizada	O(n) esperado*	O(n)	Busca e inserção em set
Fallback	$O(n^2)$	O(1) adicional	Usado para itens não hashable

* A análise O(n) é esperada para elementos hashable e distribuição usual de hashes; colisões adversas podem degradar o desempenho.

Fonte: Elaborado pelos autores (2026).

6. ESTRATÉGIA DE VALIDAÇÃO

Foram criados testes automatizados para listas vazias, listas com um elemento, listas sem repetição, duplicatas no início e no final, elementos não hashable e 300 entradas aleatórias geradas com semente fixa. Os testes comparam o retorno das duas funções. Os oito testes executados foram aprovados.

7. METODOLOGIA DE BENCHMARK

As medições foram realizadas com time.perf_counter(), cinco repetições por função e entrada. Para cada tamanho, foi gerada uma lista sem duplicatas usando random.sample() com semente determinística. A geração dos dados ocorre fora da região cronometrada. Esse cenário força a versão original a examinar todos os pares, representando seu pior caso. A tabela apresenta a mediana dos cinco tempos; o script também salva o menor tempo e cada execução no arquivo JSON.

Ambiente medido: Python 3.13.5; plataforma Linux-6.18.57-x86_64-with-glibc2.41; processador reportado pelo ambiente: AMD EPYC 9V74. Os resultados são específicos deste ambiente e podem variar em outra máquina.

8. RESULTADOS

Tabela 2 – Medianas dos tempos de execução nas listas sem as duplicatas:

n	Original (ms)	Otimizada (ms)	Ganho (x)
500	3.6821	0.0226	162.97x
1.000	15.7239	0.0425	369.59x
2.000	63.3170	0.0979	646.83x
4.000	248.8463	0.1897	1311.82x
8.000	982.6505	0.4188	2346.34x

9. ANÁLISE DOS RESULTADOS

Em n = 8.000, a versão original apresentou mediana de 982.651 ms, enquanto a versão otimizada apresentou 0.419 ms. Nesse maior tamanho testado, a razão entre as medianas foi de 2346.34x. O ganho cresceu com os tamanhos avaliados, comportamento coerente com a diferença entre crescimento quadrático e linear esperado.

A comparação em n = 500 também favoreceu a versão com set (162.97x pela mediana). Em listas pequenas, a diferença absoluta é bem reduzida; em listas maiores, a quantidade de comparações evitadas pela estrutura de hashing torna-se mais relevante.

10. TRADE-OFFS E LIMITAÇÕES

Tempo versus espaço: a versão otimizada troca memória adicional O(n) pelo menor tempo esperado O(n).

Hashabilidade: set exige elementos hashable. O fallback mantém então a correção para outros elementos, porém não preserva o ganho de desempenho nesse caso em especifico.

Colisões: a complexidade linear é esperada, não uma garantia para toda distribuição possível de hashes.

Benchmark: os tempos dependem de hardware, versão do Python, carga do sistema e tamanho da entrada. A medição deve ser repetida no computador da apresentação.

Escopo: esta é a primeira otimização da AV2; otimizações adicionais, análise mais ampla e discussão final estão previstas para a AV3.

11. CONCLUSÃO

A estrutura set reduz a quantidade de comparações necessárias e transforma o processamento do caso comum com elementos hashable de quadrático para linear esperado. Os testes indicaram equivalência funcional nos cenários avaliados e o benchmark mostrou redução expressiva do tempo para entradas crescentes. A escolha é adequada quando o objetivo é detectar duplicatas e existe memória disponível para armazenar os elementos já visitados.

12. DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Durante a elaboração deste trabalho, o grupo utilizou ferramentas de inteligência artificial generativa como apoio na organização do código, na estruturação dos testes e na revisão do texto. Todo o conteúdo foi revisado pelos integrantes, que compreendem e são capazes de explicar as soluções, as análises e os resultados apresentados.